# Operating Systems Lab

## Mini-Project Report

### Online Retail Store Management System

(Implemented in C using Linux Systems Programming)



| Roll Number | Name |
|---|---|
| BT2024038 | Sushmit Biswas |



Course: EGC 301P

# 1 Problem Statement

The objective of this project is to develop a working **Online Retail Store Management System** using Linux systems programming. To meet the OS Lab requirements, the app uses direct system calls and core concepts instead of high-level frameworks. Instead of using a database like MySQL, the system stores data in flat files, uses POSIX threads for concurrency, and implements IPC for background tasks.

The system allows Admins and Customers to log in via a TCP client, browse products, manage stock, and place orders.

# 2 Implementation of Mandatory OS Concepts

The project thoroughly implements the six required Operating Systems concepts, utilizing appropriate POSIX APIs to ensure safe, concurrent execution.

## 2.1 Role-Based Authorization

The system uses Role-Based Access Control (RBAC). Users are assigned either `ROLE_ADMIN` or `ROLE_CUSTOMER`. The server checks the role of the user before running any commands. Admin features like `ADD_PRODUCT` or `MODIFY_PRODUCT` are protected by a `require_admin()` function, which blocks regular customers from making those changes.

## 2.2 File Locking

Instead of using a database, the application stores data in simple CSV files (`users.csv`, `products.csv`, `sales.csv`). To keep file reading and writing safe, the project uses POSIX file locking with the `fcntl()` function. Before a file is opened, a `struct flock` lock is applied (using `F_RDLCK` or `F_WRLCK`) to stop other processes from overwriting it. The lock is removed with `F_UNLCK` when the file is closed.

## 2.3 Concurrency Control

The server uses multithreading to handle multiple clients at once using the POSIX Threads (`pthreads`) library. Every time a new client connects, a new thread is created using `pthread_create`. To stop threads from messing up shared memory, `pthread_mutex_t` locks are used. For example, `g_db_mutex` makes sure that only one thread can modify the product array or user list at a time.

## 2.4 Data Consistency

Managing a retail inventory can lead to race conditions. For example, if two clients try to buy the last item at the exact same time, only one should succeed. Data consistency is maintained by keeping all stock checks and order generation inside a locked section. By combining `fcntl()` and `pthread_mutex`, we make sure that dirty reads and lost updates don't happen to `data/products.csv`.

## 2.5 Socket Programming

The application uses a standard Client-Server architecture built with TCP Sockets (`<sys/socket.h>`, `<netinet/in.h>`). The `retail_server` program binds to a port and listens for incoming connections. The `retail_client` program connects to it and sends requests using `send()` and `recv()`. This allows multiple clients from different terminal windows to connect and interact with the store at the same time.

## 2.6 Inter-Process Communication (IPC)

To handle separate tasks without blocking the main program, like simulating a payment gateway, the server creates child processes using `fork()`. The parent thread and the child process communicate using `pipe()`. The child process handles the payment logic, adds a simulated delay, and then sends the status (`APPROVED` or `DECLINED`) back through the pipe so the parent thread can finish the order.

## 2.7 Automated Concurrency Verification

To test if the concurrency control works, I wrote an automated script using `tmux` and `auto_client.py`. The script starts one server and three clients at once. The clients automatically log in and place orders at the exact same time, which shows that the system handles multiple requests and updates stock correctly.

# 3 Challenges Faced and Solutions

1. **Handling File Locks properly**: Using `fcntl()` for file locking was tricky when combining it with standard `fopen()` functions. *Solution:* I created simple wrapper functions (`acquire_file_lock` and `release_file_lock`) to handle the locking safely before reading or writing to the files.

2. **Zombie Processes**: The `fork()` calls for the payment process were leaving behind zombie processes when clients disconnected. *Solution:* I used `waitpid` to make sure the parent thread properly cleans up the child processes after they finish.

# 4 Sample System Output and Features

The finalized system successfully integrates standard operational features alongside robust automated testing to prove network concurrency and thread safety. Below are the execution screenshots demonstrating the core functionalities of the application.

## 4.1 System Compilation and Setup

The project utilizes a `Makefile` for clean compilation. The server initializes and binds to the specified port, ready to accept client connections.

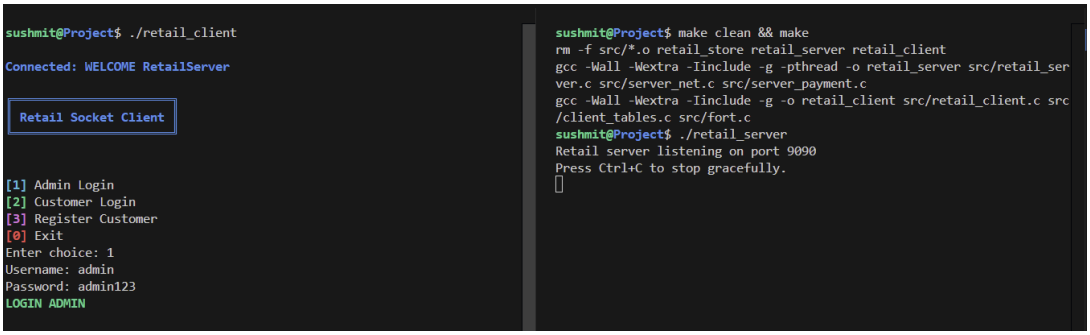



Figure 1: Clean compilation using `make` and initialization of the Retail Server and Client.

## 4.2 Administrative Functionality

The Admin role has exclusive access to inventory management, system logs, and global order tracking. The terminal UI utilizes the `libfort` library to render clean, formatted tables.

```
Retail Client (Admin)

[1] Add Product
[2] Modify Product
[3] Delete Product
[4] List Products
[5] View All Orders
[6] Update Order Status
[7] View Sales Report
[8] Create Admin
[9] View Logs
[0] Logout

Enter choice: 4

PRODUCTS 26

Available categories: books, clothing, electronics, furniture, grocery
Type 'all' to show everything.
Select category: all

--- Page 1 of 3 (Products 1-10 of 26) ---
```

| ID | NAME | CATEGORY | PRICE | STOCK |
|---|---|---|---|---|
| 1 | Aashirvaad Atta 5kg | grocery | ₹285.00 | 0 |
| 2 | Amul Butter 500g | grocery | ₹255.00 | 30 |
| 3 | Tata Salt 1kg | grocery | ₹45.00 | 100 |
| 4 | Maggi Milk Chocolate | grocery | ₹20.00 | 150 |
| 5 | Fortune Basmati Rice 5kg | grocery | ₹499.00 | 20 |
| 6 | Nescafe Maggi Noodles 140g | grocery | ₹30.00 | 200 |
| 7 | Haldirams Bhujia 400g | grocery | ₹115.00 | 40 |
| 8 | Everest Sunflower Oil 1L | grocery | ₹165.00 | 60 |
| 9 | Brooke Bond Coffee 50g | grocery | ₹150.00 | 35 |
| 10 | Lays Magic Masala 50g | grocery | ₹20.00 | 250 |

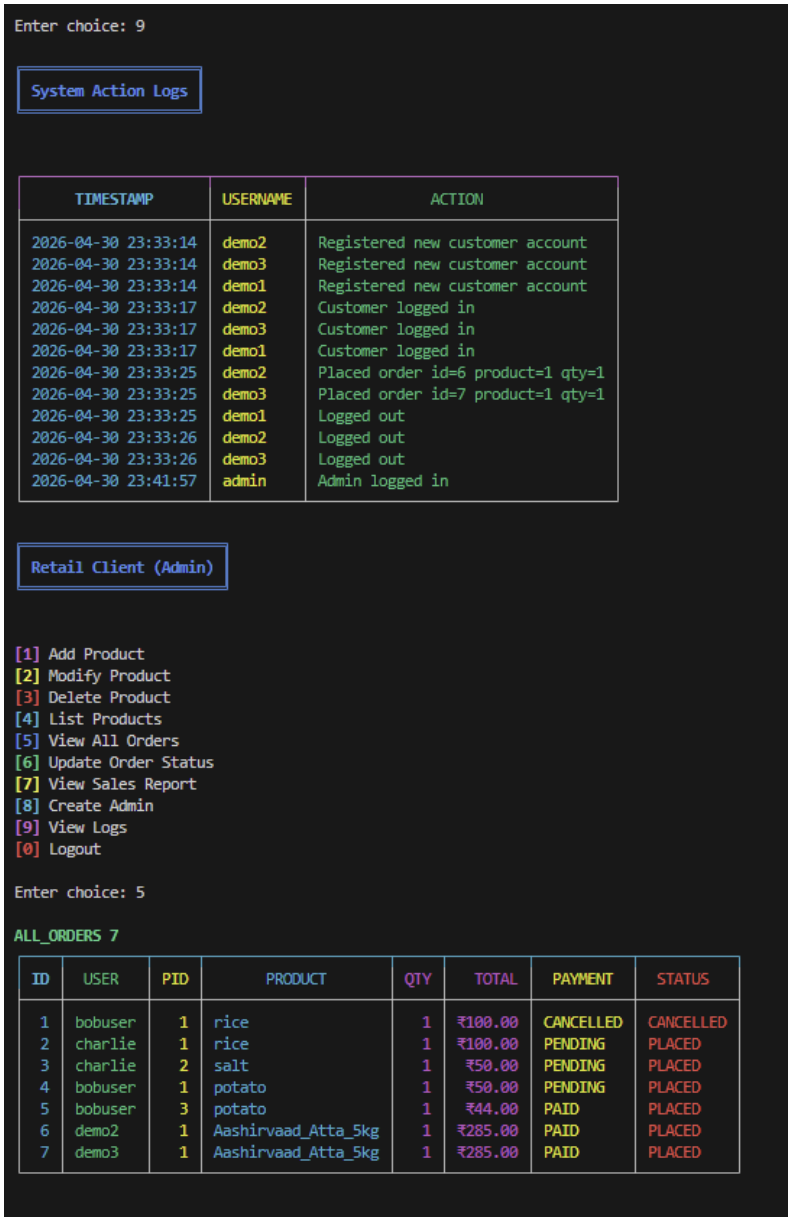
```
Enter 'n' (next 10), 'p' (prev 10), 'q' (exit to menu): n

--- Page 2 of 3 (Products 11-20 of 26) ---
```

| ID | NAME | CATEGORY | PRICE | STOCK |
|---|---|---|---|---|
| 11 | Levis Mens Jeans | clothing | ₹2199.00 | 15 |
| 12 | Puma Cotton TShirt | clothing | ₹799.00 | 25 |
| 13 | Wildcraft Winter Jacket | clothing | ₹1850.00 | 10 |
| 14 | Adidas Running Shoes | clothing | ₹3500.00 | 12 |
| 15 | Nike Sports Cap | clothing | ₹599.00 | 30 |
| 16 | Titan Analog Watch | electronics | ₹4500.00 | 8 |
| 17 | Sony WH1000XM4 Headphones | electronics | ₹22990.00 | 4 |
| 18 | Wakefit Orthopedic Mattress | furniture | ₹8499.00 | 10 |
| 19 | Nilkamal Office Table | furniture | ₹3200.00 | 12 |
| 20 | Godrej Interio Chair | furniture | ₹1550.00 | 20 |

```
Enter 'n' (next 10), 'p' (prev 10), 'q' (exit to menu): q
```

Figure 2: Admin view displaying the paginated product inventory.



```
Enter choice: 9

System Action Logs
```

| TIMESTAMP | USERNAME | ACTION |
|---|---|---|
| 2026-04-30 23:33:14 | demo2 | Registered new customer account |
| 2026-04-30 23:33:14 | demo3 | Registered new customer account |
| 2026-04-30 23:33:14 | demo1 | Registered new customer account |
| 2026-04-30 23:33:17 | demo2 | Customer logged in |
| 2026-04-30 23:33:17 | demo3 | Customer logged in |
| 2026-04-30 23:33:17 | demo1 | Customer logged in |
| 2026-04-30 23:33:25 | demo2 | Placed order id=6 product=1 qty=1 |
| 2026-04-30 23:33:25 | demo3 | Placed order id=7 product=1 qty=1 |
| 2026-04-30 23:33:25 | demo1 | Logged out |
| 2026-04-30 23:33:26 | demo2 | Logged out |
| 2026-04-30 23:33:26 | demo3 | Logged out |
| 2026-04-30 23:41:57 | admin | Admin logged in |

```
Retail Client (Admin)

[1] Add Product
[2] Modify Product
[3] Delete Product
[4] List Products
[5] View All Orders
[6] Update Order Status
[7] View Sales Report
[8] Create Admin
[9] View Logs
[0] Logout

Enter choice: 5

ALL_ORDERS 7
```

| ID | USER | PID | PRODUCT | QTY | TOTAL | PAYMENT | STATUS |
|---|---|---|---|---|---|---|---|
| 1 | bobuser | 1 | rice | 1 | ₹100.00 | CANCELLED | CANCELLED |
| 2 | charlie | 1 | rice | 1 | ₹100.00 | PENDING | PLACED |
| 3 | charlie | 2 | salt | 1 | ₹50.00 | PENDING | PLACED |
| 4 | bobuser | 1 | potato | 1 | ₹50.00 | PENDING | PLACED |
| 5 | bobuser | 3 | potato | 1 | ₹44.00 | PAID | PLACED |
| 6 | demo2 | 1 | Aashirvaad_Atta_5kg | 1 | ₹285.00 | PAID | PLACED |
| 7 | demo3 | 1 | Aashirvaad_Atta_5kg | 1 | ₹285.00 | PAID | PLACED |

Figure 3: Admin interface showing real-time system action logs and global user orders.

## 4.3 Customer Operations

Customers can securely log in, browse available stock, execute purchases via different payment methods, and view their individual transaction history.

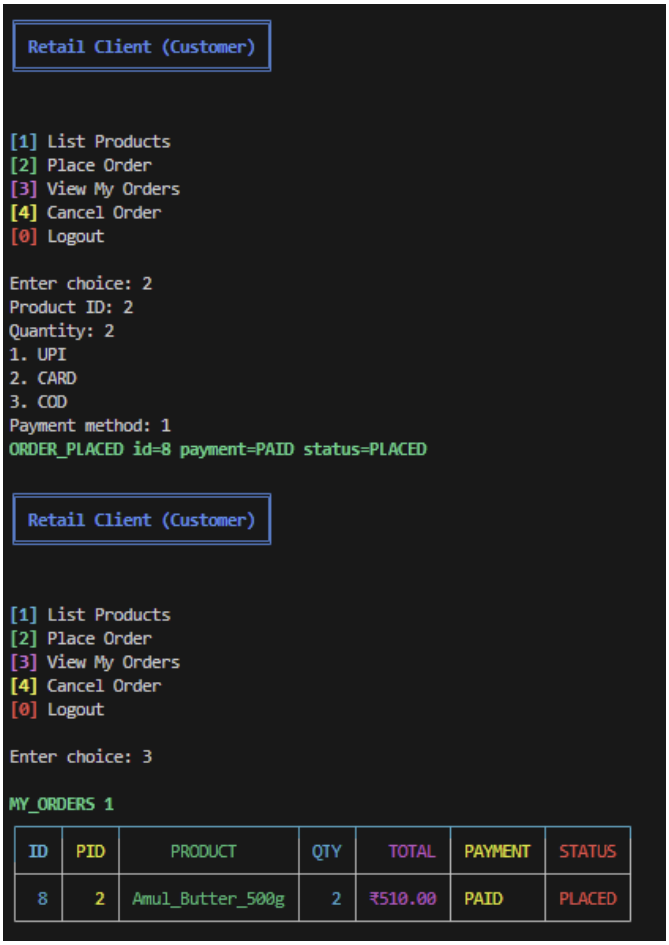



Figure 4: Customer placing an order with UPI and verifying the transaction in their order history.

## 4.4 Concurrency and Thread Safety

To demonstrate race-condition prevention and POSIX thread safety under load, an automated testing suite was integrated into the workflow:

- `auto_client.py`: A Python automation script that interfaces directly with the compiled C client binary. It programmatically simulates a rapid, end-to-end user workflow (account registration, login, stock purchasing, and fetching order history).
- `demo_concurrency_tmux.sh`: A bash orchestrator that utilizes a `tmux` environment to spin up the central server in one pane and simultaneously trigger multiple Python automated clients in split panes.

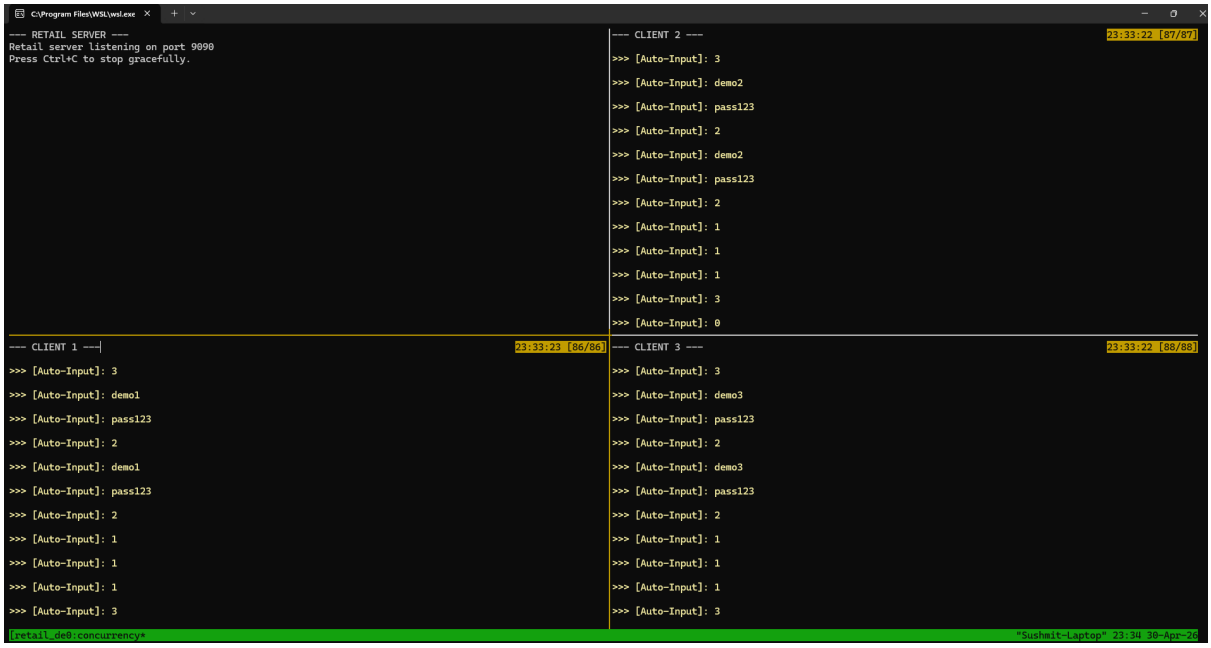



Figure 5: Tmux environment orchestrating three automated clients firing simultaneous requests to the central server.

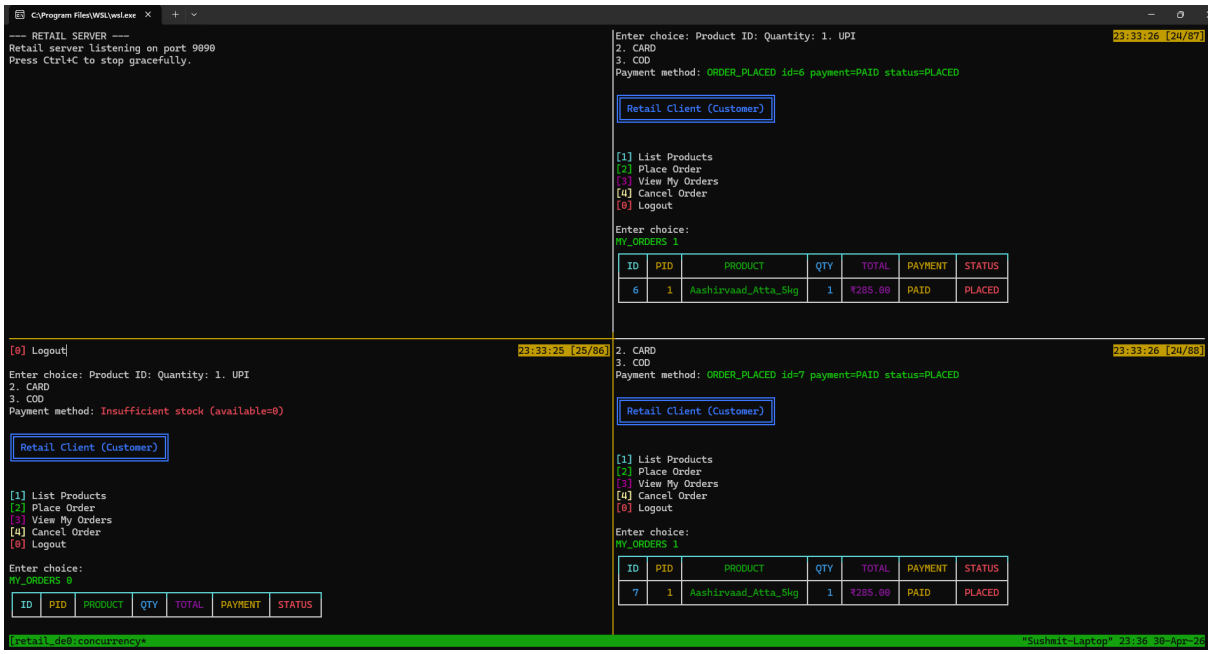



Figure 6: Concurrency in action: The thread-safe architecture correctly processes simultaneous purchases. Client 1's transaction is safely rejected due to "Insufficient stock" after Clients 2 and 3 consumed the remaining inventory, proving data consistency and active file locking.